PRODUCT REQUIREMENTS DOCUMENT

Go-To Recipes

A personal recipe library with smart search, filtering, and AI-powered link summaries

Document Version	1.0 (Draft)
Date	March 1, 2026
Author	[Your Name]
Status	Draft — Ready for Review
Target Platform	Web (responsive)

Table of Contents

1. Overview

Go-To Recipes is a personal web application for storing, organizing, and quickly finding favorite recipes. It supports two recipe types: linked recipes that reference external URLs with AI-generated summaries, and native recipes with full ingredients and steps entered directly. The app prioritizes speed of browsing through powerful filtering, search, and “go-to” signals like ratings and cook counts.

1.1 Problem Statement

Home cooks accumulate recipes across dozens of bookmarks, screenshots, notes apps, and recipe sites. There is no single, personal, fast-to-browse hub that combines external recipe links with personal notes and makes it easy to answer “What should I cook tonight?” based on season, time, cuisine, or mood.

1.2 Product Vision

A clean, fast, personal recipe library that feels like flipping through a well-organized cookbook — but with the power of search, smart filters, and AI summaries so you never lose a great recipe again.

1.3 Target User

Single-user (personal/household) for MVP. The primary user is a home cook who collects recipes from many sources and wants a single place to organize, tag, rate, and quickly find them.

1.4 Success Metrics

- **Adoption:** User adds 20+ recipes within the first two weeks of use.
- **Engagement:** User opens the app 3+ times per week to find a recipe.
- **AI value:** 80%+ of linked recipe summaries are accepted without manual editing.
- **Speed:** User can find a relevant recipe in under 15 seconds using filters/search.

---

2. Core Features (MVP)

2.1 Recipe Library

Users can create, edit, and delete recipes. Each recipe belongs to one of two types:

2.1.1 Linked Recipes

Stores an external URL as the canonical source. The app fetches the page content server-side and uses an LLM to generate a short summary, bullet highlights, and a best-effort ingredients list. The original link is always prominently displayed. Users can add personal notes on top of the generated content.

2.1.2 Native Recipes

Full recipe content entered directly into the app: ingredients list, step-by-step instructions, and personal notes. No external URL required.

2.1.3 Recipe Card View

The library displays recipes as scannable cards showing: title, cuisine, dish type, “good for” tags, season tags, total time, and a short description. Users can toggle between grid and list layouts.

2.2 Filtering and Search

Finding the right recipe quickly is the core UX promise. The system provides full-text search and faceted filters that can be combined.

2.2.1 Full-Text Search

Searches across recipe title, personal notes, ingredients, and AI-generated summaries. Results should appear as the user types (debounced).

2.2.2 Faceted Filters

All filters support multi-select where appropriate. Filters combine with AND logic across categories and OR logic within a category.

Filter	Options	Notes
Season	Spring, Summer, Fall, Winter, Any	Multi-select
Dish Type	Appetizer, Main, Side, Dessert, Snack, Breakfast/Brunch, Soup/Stew, Salad, Drink/Cocktail	Multi-select
Cuisine	Italian, Mexican, Indian, Thai, American, Mediterranean, etc.	Multi-select + custom values allowed
Good For	Weeknight, Dinner Party, Meal Prep, Date Night, Kid-Friendly, Crowd/Potluck, Healthy-ish, Comfort Food	Multi-select
Dietary	Vegetarian, Vegan, GF, DF, Nut-free, etc.	Multi-select; optional on recipes
Time	<30 min, 30–60 min, 60+ min	Based on totalTimeMinutes
Protein / Main Ingredient	Chicken, Beef, Fish, Tofu, etc.	Optional; multi-select

2.2.3 Sort Options

- Recently Added (default)

- Most Cooked (descending cook count)
- Highest Rated (descending rating)
- Prep Time (ascending)

2.3 “Go-To” Signals

These fields help surface the user’s most trusted and frequently used recipes, enabling smarter sorting and future recommendation features.

Signal	Type	Behavior
Rating	1–5 stars (nullable)	Set or updated by user on recipe detail page
Cook Count	Integer, default 0	Incremented via a “Cooked it!” button on recipe detail
Favorite / Pin	Boolean	Toggle to pin recipe; pinned recipes appear first in default sort
Last Cooked	Date (nullable)	Auto-set when cook count is incremented; can be edited

---

2.4 AI Agent: URL Synthesis

When a user creates a linked recipe, the system uses an LLM to generate structured summary data from the recipe URL. This is the key differentiator that makes linked recipes useful without leaving the app.

2.4.1 Workflow

1. User enters a URL and submits.
2. Server fetches the page content (HTML) from the URL.
3. Server sends extracted text to LLM with a structured prompt.
4. LLM returns: short description (1–2 sentences), bullet highlights (time/effort, techniques, standout flavors), and ingredients list (best effort).
5. User reviews and edits the generated fields before saving.

2.4.2 Reliability and Safety

- **Fallback:** If the URL fetch fails or the page is paywalled, prompt the user to paste the recipe text manually.
- **Caching:** Store generated summaries so repeated views don’t re-trigger the LLM. Include a “Regenerate” button.
- **Transparency:** Display “Generated from link on [date]” alongside the summary.

- **Attribution:** Always keep the canonical external link prominently accessible. The app must never appear to “steal” recipe content from the source.

---

3. Pages and UX Flows

3.1 Home / Library Page

The primary screen. Displays a grid (or list) of recipe cards with a persistent search bar at the top and a collapsible filter drawer (sidebar or dropdown panel). Active filters are shown as removable chips. The page should feel fast and responsive — filtering and search happen client-side where possible.

3.2 Recipe Detail Page

Displays full recipe information organized by type:

- **Common elements:** Title, tags (season, cuisine, dish type, good-for, dietary), rating stars, cook count with “Cooked it!” button, favorite toggle, and personal notes.
- **Linked recipe:** AI-generated short description, bullet highlights, best-effort ingredients, and a prominent “Open Original Recipe” button linking to the source URL.
- **Native recipe:** Full ingredients list and step-by-step instructions.

3.3 Add / Edit Recipe Flow

The user first chooses between Linked and Native recipe types. The flow then diverges:

Linked: Enter URL → System generates AI summary → User reviews/edits generated fields and tags → Save.

Native: Manual entry form for title, ingredients, steps, tags, and notes → Save.

Both flows share common tag/metadata fields (season, cuisine, dish type, good-for, dietary, time, protein). All tag fields should support both preset options and custom free-text entry where noted.

---

4. Data Model

The following schema defines the core entities. Field types use standard conventions (text, int, bool, datetime, array of strings). The schema should be implemented in SQLite for local development and be compatible with Postgres for hosted deployment.

4.1 Recipe

Field	Type	Description
id	UUID / auto-increment	Primary key
title	text, required	Recipe name
recipeType	enum: linked | native	Determines which fields are active
url	text, nullable	External recipe URL (linked type only)
descriptionShort	text, nullable	1–2 sentence summary (AI-generated or manual)
highlights	text[] / JSON	Bullet-point highlights from AI or user
ingredients	text[] / JSON	Ingredients list (structured or string array)
steps	text[] / JSON	Cooking steps (native type)
personalNotes	text, nullable	User's personal notes
seasonTags	text[] / JSON	e.g., [Summer, Fall]
dishTypes	text[] / JSON	e.g., [Main, Side]
cuisineTypes	text[] / JSON	e.g., [Italian, Mediterranean]
goodForTags	text[] / JSON	e.g., [Weeknight, Meal Prep]
dietaryTags	text[] / JSON	e.g., [Vegetarian, GF]
mainIngredientTags	text[] / JSON	e.g., [Chicken, Lemon]
prepTimeMinutes	int, nullable	Prep time in minutes
cookTimeMinutes	int, nullable	Cook time in minutes
totalTimeMinutes	int, nullable	Total time (derived or manual)
servings	text, nullable	e.g., "4" or "4–6"
rating	int 1–5, nullable	User star rating
isFavorite	bool, default false	Pinned / favorite flag
cookCount	int, default 0	Times cooked
lastCookedAt	datetime, nullable	Last cooked timestamp
createdAt	datetime	Auto-set on creation
updatedAt	datetime	Auto-set on update

4.2 AI Summary Job

Tracks the status and output of AI summarization requests for linked recipes.

Field	Type	Description

id	UUID / auto-increment	Primary key
recipeId	FK → Recipe	Associated recipe
status	enum: pending | processing | completed | failed	Job lifecycle state
sourceUrl	text	URL that was fetched
fetchedAt	datetime, nullable	When the URL was fetched
modelUsed	text, nullable	LLM model identifier
rawExtract	text, nullable	Raw text extracted from URL (optional, for debugging)
errorMessage	text, nullable	Error details if job failed
createdAt	datetime	Auto-set

---

5. Technical Requirements

5.1 Stack Recommendations

The following stack is recommended for a pragmatic, modern MVP that balances developer experience with ease of deployment. The final stack choice should be validated during implementation.

Layer	Recommendation	Rationale
Frontend	Next.js (App Router) + Tailwind CSS	Full-stack React framework with SSR, file-based routing, and API routes in one project
Backend / API	Next.js API Routes or Server Actions	Co-located with frontend; no separate server needed for MVP
Database	SQLite (local) / Postgres (hosted)	SQLite for zero-config dev; Postgres for production via Vercel Postgres, Supabase, or Neon
ORM	Prisma or Drizzle	Type-safe schema, migrations, and queries
AI / LLM	Anthropic Claude API (or OpenAI)	Structured output for recipe summarization
URL Fetching	Server-side fetch + HTML-to-text extraction	Cheerio or Mozilla Readability for clean text extraction
Deployment	Vercel (primary) or Fly.io	Zero-config Next.js deployment with edge functions

5.2 Non-Functional Requirements

- **Performance:** Library page loads in under 2 seconds with 200+ recipes. Filters apply instantly (client-side where possible).
- **Responsiveness:** Fully usable on mobile, tablet, and desktop. Mobile-first design for kitchen use.
- **Authentication:** Optional for MVP (single-user assumed). Design data model and API so auth can be layered in later without restructuring.
- **Data persistence:** All recipe data persisted to database. No data loss on page refresh or app restart.
- **Testing:** Unit tests for critical logic: filtering, search indexing, and AI summarization endpoint.

5.3 Environment Variables

The application requires the following environment variables for configuration:

- DATABASE_URL — Connection string for SQLite or Postgres
- ANTHROPIC_API_KEY (or OPENAI_API_KEY) — LLM provider credentials for AI summarization
- NEXT_PUBLIC_BASE_URL — Base URL for the deployed app (optional, for link generation)

---

6. Scope and Constraints

6.1 MVP Scope (In)

- Full CRUD for linked and native recipes
- AI-powered URL summarization with caching and manual fallback
- Full-text search and all faceted filters listed in Section 2.2
- Go-to signals: rating, cook count, favorite, last cooked
- Responsive card-based library view with grid/list toggle
- Recipe detail page with type-specific display
- Seed data with 5–10 example recipes for demonstration

6.2 Out of Scope (Post-MVP)

- User authentication and multi-user support
- Meal planning and calendar integration
- Grocery list generation from selected recipes
- Social features (sharing, public profiles)
- Image upload and recipe photo management
- Browser extension for one-click recipe saving
- Import from other recipe apps or bookmarks

6.3 Key Constraints

- **Attribution first:** For linked recipes, the app stores only summaries and metadata. The canonical external link is always prominently displayed. The app must never appear to claim or reproduce full recipe content from external sources.
- **Keep it shippable:** Resist gold-plating. Focus on core search/filter/browse UX over visual polish.
- **Single project:** Frontend, API, and database in one deployable unit for MVP simplicity.

---

7. Open Questions

#	Question	Impact
1	Should the app support image upload for recipes in MVP, or defer to post-MVP?	Scope / storage cost
2	What LLM provider to use? Claude vs. OpenAI vs. open-source? Budget considerations?	Cost / implementation
3	Should tag values (cuisine, dietary, etc.) be fully user-defined or start from a fixed list with custom additions?	Schema / UX design
4	Is offline/PWA support needed for kitchen use (no Wi-Fi)?	Architecture
5	Should there be any sharing capability (e.g., share a recipe link with a partner)?	Scope

---

8. Appendix

8.1 User Story Map

Priority user stories for MVP implementation, ordered by importance:

1. As a user, I want to add a recipe by URL so the app generates a summary I can browse without leaving the app.
2. As a user, I want to search and filter my recipes by cuisine, season, dish type, and time so I can quickly find what to cook.
3. As a user, I want to rate and mark recipes as favorites so my best recipes surface first.
4. As a user, I want to add a native recipe with ingredients and steps for my original recipes.
5. As a user, I want to track how many times I've cooked a recipe so I know my true go-tos.
6. As a user, I want to click through to the original recipe URL so I can see the full source.

8.2 Glossary

- **Linked Recipe:** A recipe entry that references an external URL as its canonical source, with an AI-generated summary stored locally.
- **Native Recipe:** A recipe entry with full ingredients and instructions stored directly in the app.
- **Go-To Signal:** Metadata (rating, cook count, favorite status) that indicates how trusted and frequently used a recipe is.
- **Faceted Filter:** A filter category (e.g., cuisine, season) that allows selecting one or more values to narrow results.
- **AI Summary Job:** A background process that fetches a URL, extracts content, and uses an LLM to generate structured recipe metadata.